



TRAKYA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM411 YAPAY SİNİR AĞLARINA GİRİŞ
PROJE RAPORU

PROJE ADI:

Yaşanabilirlik Odaklı Güvenlik Skoru ve Suç Analiz Sistemi

PROJE EKİBİ:

1221602005 – Duygu TAKTAK

1221602015- Helin YILMAZ

1221602079- Berfin KILINÇ

1221602093 – Azra AKTAŞ

1231602805 – Beliz BİNYAL

PROJE DANIŞMANI:

DR. ÖĞR. ÜYESİ TURGUT DOĞAN

Edirne - 2026

İÇİNDEKİLER

1. GİRİŞ & PROJENİN TANITIMI	2
2. GEREKSİNİM ANALİZİ	3
2.1. Platform ve Teknik Gereksinimler	4
2.2. Kaynak Kısıtları ve Çözümler	4
2.3. Mevcut Sistemlerden Farkı	4
2.4. Hedef Kitle	4
2.5. Teknoloji Yığılı	4
3. TASARIM	5
3.1. Sistem Mimarisi	6
3.2. Güvenlik Skoru Algoritması	6
3.3. K-Means Kümeleme.....	6
3.4. Kullanıcı Arayüzü	7
4. GELİŞTİRME	7
4.1. Güvenlik Skoru Hesaplama.....	7
4.2. Model Karşılaştırması	7
4.3. Çalıştırma Yönergeleri ve Sistem Konfigürasyonu.....	7
5. TEST ve DOĞRULAMA.....	7
5.1. Yazılım ve Veri Bütünlüğü Doğrulaması.....	7
5.2. Makine Öğrenmesi Model Testleri ve Sonuçların Değerlendirilmesi.....	7
6. SONUÇ	7
KAYNAKLAR.....	7

1. GİRİŞ & PROJENİN TANITIMI

Modern şehir yönetiminde ve yaşanabilirlik (livability) standartlarının belirlenmesinde bölgesel güvenlik, en kritik belirleyicilerin başında gelmektedir. Geleneksel güvenlik ve suç analizleri genellikle bölgelerdeki toplam suç vakalarının basit bir frekans dağılımı üzerinden yapılmaktadır. Ancak bir mahallenin gerçek risk profilini ve yaşanabilirlik seviyesini yalnızca toplam suç sayısı ile ölçmek, yanıltıcı sonuçlar doğurabilmektedir. Suçun türü, şiddet ağırlığı, gerçekleştiği zaman dilimi ve lokasyon gibi bağlamsal faktörlerin göz ardı edilmesi, risk analizinin doğruluğunu düşürmektedir. Bu bağlamda, ham verinin anlamlı bir metriğe dönüştürülmesini sağlayacak bütüncül ve analitik bir sisteme ihtiyaç duyulmaktadır.

Yaşanabilirlik Odaklı Güvenlik Skoru ve Suç Analiz Sistemi projesinin temel amacı; suç verilerini çok boyutlu bir yaklaşımla inceleyerek her bir bölge için 0 ile 100 arasında normalize edilmiş, objektif bir güvenlik skoru (safety score) üretmektir. Bu doğrultuda proje, yalnızca vaka sayılarını toplamak yerine belirli suç türlerine ve bu suçların şiddet ağırlıklarına göre risk katsayıları oluşturmayı hedeflemektedir. Elde edilen bu analitik skorlar kullanılarak bölgeler, K-Means kümeleme algoritması yardımıyla risk seviyelerine göre anlamlı segmentlere ayrılmaktadır.

Projede, LSTM derin öğrenme mimarisi kullanılarak geleceğe yönelik bölgesel güvenlik skorları tahmin edilmiş ve bu skorlar üzerinden kullanıcılara rehberlik edecek anlık risk ve emniyet analizleri aktarılmıştır. Çalışma aynı zamanda, XGBoost, Random Forest ve Yapay Sinir Ağları (ANN) gibi çeşitli regresyon tabanlı makine öğrenmesi modellerinden faydalanarak bölgesel güvenlik skorlarını yüksek doğrulukla tahmin etmeyi vizyon edinmiştir. Proje, veri işlemeden modellemeye kadar uzanan tüm bu analitik süreçlerin çıktılarını, son kullanıcının kolayca anlayıp etkileşime geçebileceği interaktif haritalar ve dinamik grafiklerle desteklenen kapsamlı bir gösterge paneli (dashboard) üzerinden sunarak karar vericilere stratejik bir değerlendirme aracı sağlamayı amaçlamaktadır.

2. GEREKSİNİM ANALİZİ

2.1. Platform ve Teknik Gereksinimler

Proje, web tabanlı bir arayüz (dashboard) mimarisi üzerine inşa edildiği için sunucu, masaüstü, tablet ve mobil platformların tamamında yalnızca bir web tarayıcısı üzerinden çalışabilmektedir. Arka plan işlemleri ve modelleme süreçleri Python tabanlı geliştirildiğinden sistem; Windows, macOS ve Linux işletim sistemlerinde platformdan bağımsız (cross-platform) olarak yürütülebilme esnekliğine sahiptir.

2.2. Kaynak Kısıtları ve Çözümler

Projenin geliştirilmesi aşamasında karşılaşılan en temel kaynak kısıtı, 2001 yılından günümüze kadar uzanan devasa boyuttaki Chicago suç verisinin API üzerinden çekilmesi ve yerel bellekte işlenmesi sırasında yaşanan performans ve hız problemleridir. Bu donanımsal ve ağ tabanlı kısıtlamaları aşmak amacıyla, veri setleri ham (raw) ve işlenmiş (processed) olarak iki ayrı dizinde tutularak tekrarlı indirme işlemlerinin önüne geçilmiş, Pandas kütüphanesinin bellek optimizasyon teknikleri kullanılmış ve geliştirme aşamasında modellerin eğitilmesi için verinin tümü yerine temsil yeteneği yüksek örneklemeler (sampling) üzerinden çalışarak donanım yükü hafifletilmiştir.

2.3. Mevcut Sistemlerden Farkı

Geliştirilen bu sistem, şehir güvenliğinin ve yaşanabilirliğin yalnızca basit suç frekanslarıyla ölçülmesi problemine analitik bir çözüm getirmektedir. Mevcut uygulamalar genellikle suç olaylarını sadece harita (heatmap) üzerinde nokta bazlı haritalar şeklinde gösterirken, bu proje her bir suçun türünü ve şiddet ağırlığını matematiksel olarak hesaba katarak 0-100 arası standardize edilmiş bir güvenlik skoru üretmesiyle öne çıkmaktadır. Ayrıca, makine öğrenmesi modelleriyle gelecekteki güvenlik skorlarını tahmin edebilmesi ve K-Means algoritması ile bölgeleri risk profillerine göre akıllı segmentlere ayırması yönüyle klasik veri görselleştirme araçlarının çok ötesinde, öngörüselsel bir sistem sunmaktadır. Bu öngörüselsel altyapının en gelişmiş katmanı olarak, LSTM derin öğrenme modeli vasıtasıyla dinamik gelecek tahminleri gerçekleştirilmiş; elde edilen tahmini güvenlik skorları nitel bir yorumlama süzgecinden geçirilerek son kullanıcılara proaktif risk ve emniyet değerlendirmeleri şeklinde sunulmuştur.

2.4. Hedef Kitle

Projenin hedef kitlesi oldukça geniş bir yelpazeyi kapsamaktadır; güvenli bir bölgede yaşamak veya seyahat etmek isteyen standart vatandaşlar, bölgesel risk analizine ihtiyaç duyan gayrimenkul yatırımcıları, şehir planlamacıları ve kaynak tahsisini optimize etmeyi hedefleyen yerel güvenlik birimleri ana kullanıcıları oluşturmaktadır.

2.5. Teknoloji Yığını

Projede temel programlama dili olarak zengin kütüphane ekosistemine sahip olan Python kullanılmıştır. Veri manipülasyonu için Pandas ve NumPy; makine öğrenmesi modellerinin inşası için Scikit-learn ve yüksek performanslı XGBoost; görselleştirme adımları için ise interaktif yapıları sebebiyle Folium, Plotly ve Matplotlib tercih edilmiştir. Kullanıcı arayüzünün hızlı ve etkili bir şekilde ayağa kaldırılması için Streamlit framework'ü kullanılmıştır. Streamlit'in Python kodlarını doğrudan web arayüzüne çevirebilmesi en büyük avantajı iken, özel front-end (HTML/CSS) tasarımlarında esnekliğinin sınırlı olması bir dezavantaj olarak kabul edilebilir. Ancak veri odaklı bir gösterge paneli yaratma hedefi için en optimum çözüm olduğu için bu teknoloji yığını seçilmiştir.

Projenin kod tabanında doğrudan yer almayan ancak geliştirme sürecini destekleyen çeşitli yardımcı araçlardan da faydalanılmıştır. Bu kapsamda; bağımlılıkların izole yönetimi için Python sanal ortamları (venv) ve API test süreçleri için ağ istek araçları kullanılmıştır. Ayrıca, versiyon kontrolü ve proje yönetimi için Git altyapısı (GitHub) tercih edilerek geliştirme döngüsü güvence altına alınmıştır.

3. TASARIM

3.1. Sistem Mimarisi

Projenin sistem mimarisi, verinin kaynağından alınıp son kullanıcıya anlamlı bir arayüzle sunulmasına kadar uzanan modüler bir veri boru hattı şeklinde tasarlanmıştır. Geliştirme süreci; veri toplama, veri temizleme ve ön işleme, güvenlik skoru hesaplama, modelleme ve kullanıcı arayüzü entegrasyonu olmak üzere beş temel aşamadan oluşmaktadır. Sistem öncelikle Chicago Data Portal API'sine bağlanarak son 5 yıldan 1.185.044 satır ham suç verisini çekmekte ve data/raw dizinine kaydetmektedir. Ardından eksik verilerin giderilmesi, coğrafi koordinatların (enlem/boylam) düzenlenmesi ve zaman damgalarının anlamlı periyotlara ayrıştırılması gibi yapısal temizleme işlemleri gerçekleştirilerek veri, analiz için hazır hale getirilmektedir. Modüler klasör yapısı sayesinde her bir aşama (data_collection.py, data_cleaning.py, safety_score.py) birbirinden bağımsız çalışabilen ve test edilebilen betikler halinde kurgulanmıştır.

3.2. Güvenlik Skoru Algoritması

Projenin en kritik algoritmik adımı olan Güvenlik Skoru üretimi, her suç tipinin toplum üzerindeki etkisine göre ağırlıklandırılması mantığına dayanmaktadır. Bu aşamada sadece toplam vaka sayısı değil; cinayet ve saldırı gibi ağır suçlara yüksek ceza puanları, hırsızlık gibi suçlara ise nispeten daha düşük ceza puanları atanarak bölgesel risk değerleri hesaplanmakta ve 0-100 aralığında min-max normalizasyonu ile standartlaştırılmaktadır.

```
CRIME_WEIGHTS = {  
    "HOMICIDE": 10,  
    "CRIMINAL SEXUAL ASSAULT": 9,  
    "ROBBERY": 8,  
    "ASSAULT": 7,  
    "BATTERY": 7,  
    "WEAPONS VIOLATION": 7,  
    "BURGLARY": 6,  
    "MOTOR VEHICLE THEFT": 6,  
    "ARSON": 6,  
    "THEFT": 4,  
    "CRIMINAL DAMAGE": 4,  
    "NARCOTICS": 4,  
    "DECEPTIVE PRACTICE": 3,  
    "OTHER OFFENSE": 2,  
    "PUBLIC PEACE VIOLATION": 2  
}
```

Şekil 3.2.1. Suç Tipi Ağırlıklandırma

3.3. K-Means Kümeleme

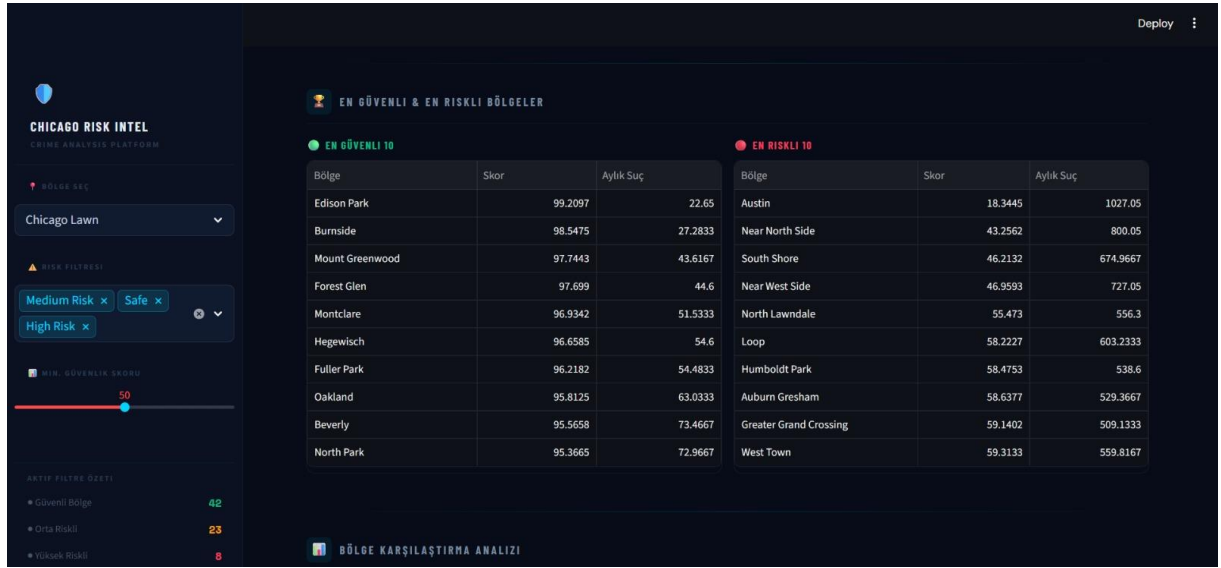
Hesaplanan güvenlik skorları üzerinden K-Means kümeleme algoritması çalıştırılarak bölgeler; "Düşük Risk", "Orta Risk" ve "Yüksek Risk" olmak üzere homojen kümelerle ayrılmaktadır. Bu denetimsiz öğrenme adımının ardından, hesaplanan güvenlik skorlarını geleceğe yönelik tahmin edebilmek amacıyla Random Forest, XGBoost ve Yapay Sinir Ağları (ANN) gibi regresyon modelleri eğitilmekte ve en düşük hata oranına (MAE/RMSE) sahip model sistemin ana tahmin motoru olarak seçilmektedir.

3.4. Kullanıcı Arayüzü

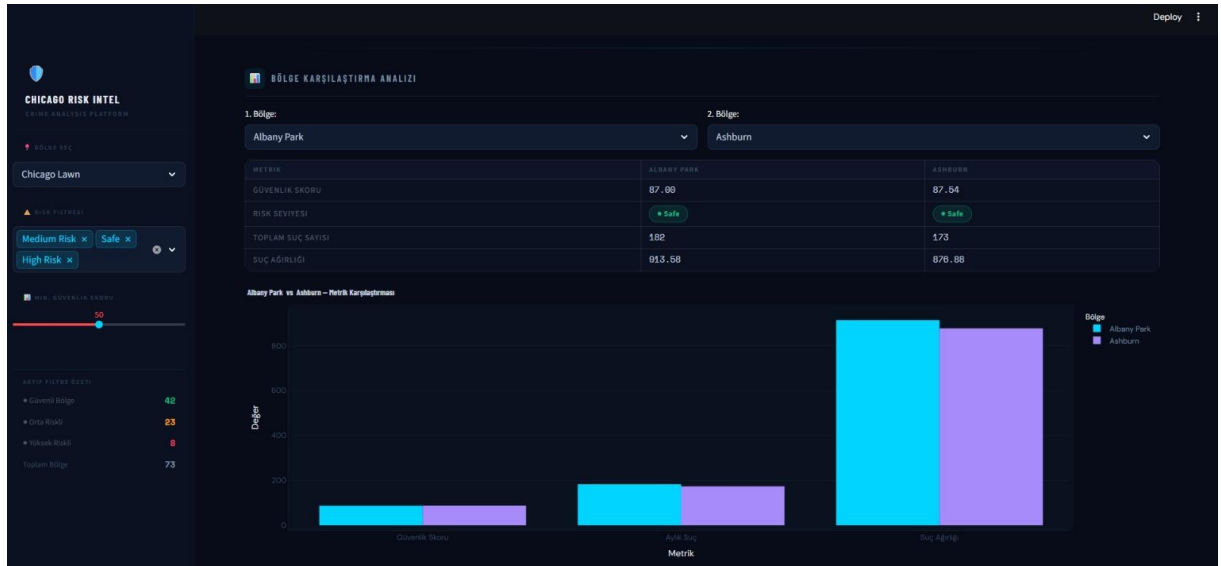
Son kullanıcı etkileşimi, arka planda çalışan tüm bu karmaşık analitik süreçleri görselleştiren Streamlit tabanlı bir web dashboard üzerinden sağlanmaktadır. Kullanıcı arayüzü; bölgelerin güvenlik skorlarını ve K-Means küme dağılımlarını gösteren Folium tabanlı interaktif risk haritalarını, suç yoğunluklarını yansıtan ısı haritalarını (heatmap) ve model performanslarının karşılaştırıldığı Plotly tabanlı dağılım grafiklerini içermektedir. Böylece sistem, hem algoritmik olarak güçlü bir makine öğrenmesi altyapısı sunmakta hem de veriyi son kullanıcının kolayca yorumlayabileceği, görsel açıdan zengin ve akıcı bir tasarımla ekrana yansıtmaktadır.



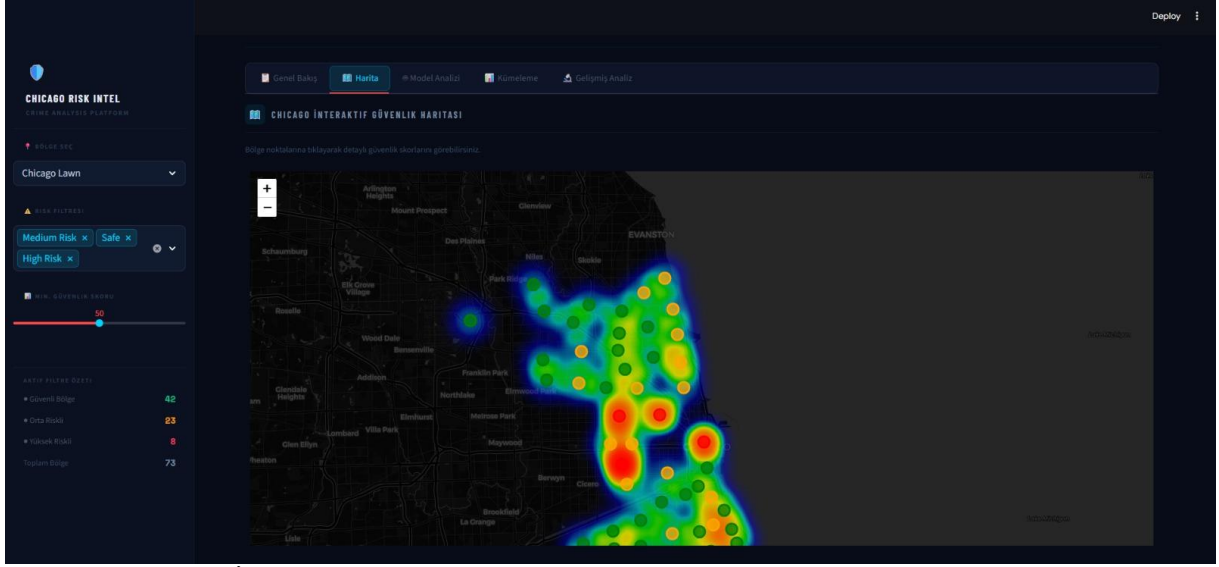
Şekil 3.4.1. Arayüz Genel Görünüşü



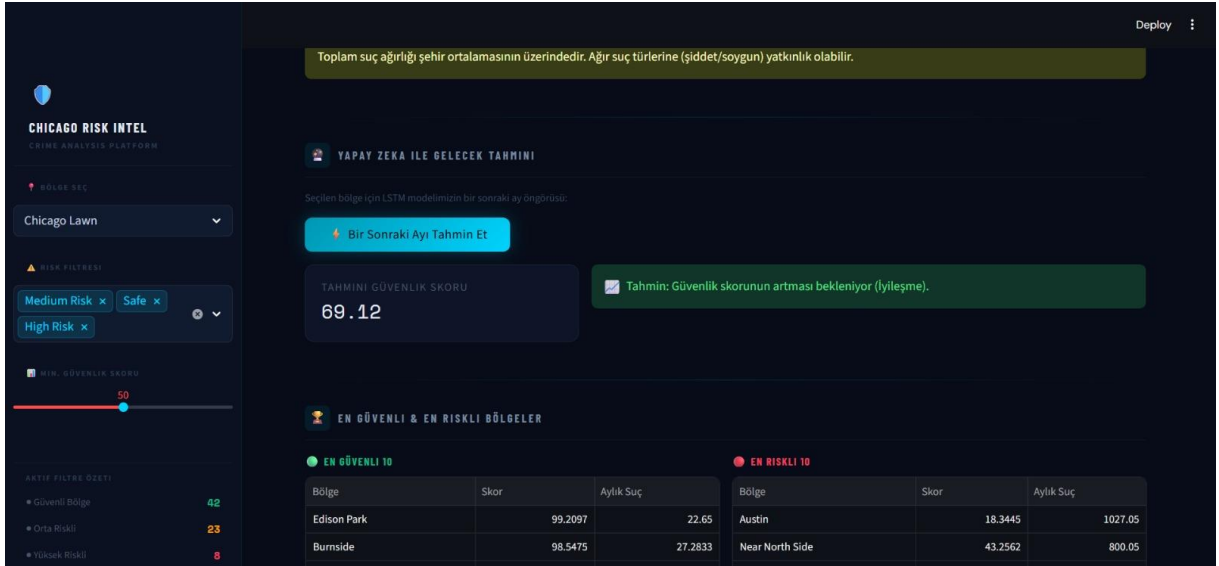
Şekil 3.4.2. En Güvenli ve En Riskli 10 Bölge



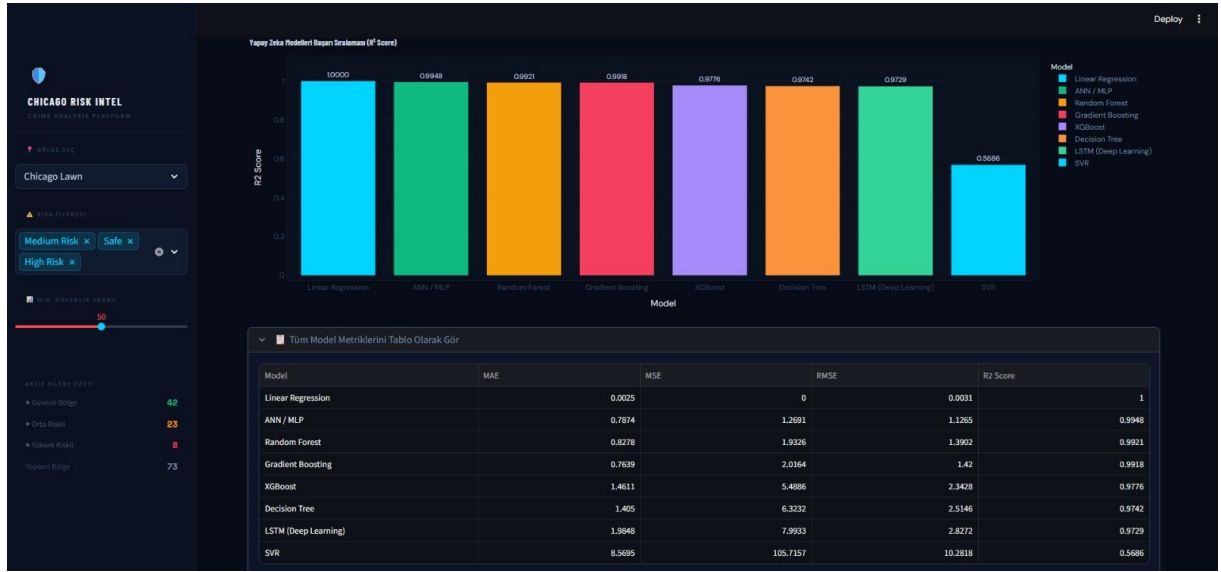
Şekil 3.4.3. Bölge Karşılaştırma Analizi



Şekil 3.4.4. İnteraktif Güvenlik Isı Haritası



Şekil 3.4.5. Gelecek Aylar İçin Güvenlik Skoru Tahmini



Şekil 3.4.6. Model Metriklerinin Karşılaştırılması

4. GELİŞTİRME

4.1. Güvenlik Skoru Hesaplama

Projenin geliştirilme sürecindeki en öğretici ve analitik açıdan en zorlu aşama, ham veriden anlamlı bir metrik elde etme süreci olmuştur. Sadece toplam suç sayılarını almak yerine, her suçun türüne göre toplumsal riskini belirleyen matematiksel bir model kurularak Güvenlik Skoru hesaplanmıştır.

Aşağıdaki kod bloğu, safety_score.py dosyasında yer alan ve kategorik suç verilerini ağırlıklandırarak Min-Max normalizasyonu ile 0-100 aralığında bir skora dönüştüren kritik algoritmayı göstermektedir.

```
def calculate_safety_score(df):
    grouped_df = df.groupby(
        ["community_area", "year", "month"]
    ).agg(
        total_crime_count=("id", "count"),
        total_crime_weight=("crime_weight", "sum"),
        avg_latitude=("latitude", "mean"),
        avg_longitude=("longitude", "mean")
    ).reset_index()

    min_weight = grouped_df["total_crime_weight"].min()
    max_weight = grouped_df["total_crime_weight"].max()

    grouped_df["risk_score"] = (
        (grouped_df["total_crime_weight"] - min_weight)
        / (max_weight - min_weight)
    ) * 100

    grouped_df["safety_score"] = 100 - grouped_df["risk_score"]

    grouped_df["safety_score"] = grouped_df["safety_score"].round(2)
    grouped_df["risk_score"] = grouped_df["risk_score"].round(2)

    return grouped_df
```

Şekil 4.1.1. Güvenlik Skoru Hesaplama

4.2. Model Karşılaştırması

Farklı makine öğrenmesi algoritmalarının (Random Forest, XGBoost, SVR vb.) aynı veri seti üzerinde eğitilerek metriklerinin (MAE, RMSE, R2) karşılaştırıldığı süreç, model seçimi ve hiperparametre optimizasyonu konularında derinlemesine bir pratik sağlamıştır.

```

results = []
all_predictions = []

for model_name, model in models.items():

    model.fit(X_train, y_train)

    predictions = model.predict(X_test)

    mae = mean_absolute_error(y_test, predictions)
    mse = mean_squared_error(y_test, predictions)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, predictions)

    results.append({
        "Model": model_name,
        "MAE": round(mae, 4),
        "MSE": round(mse, 4),
        "RMSE": round(rmse, 4),
        "R2 Score": round(r2, 4)
    })

    for actual, predicted in zip(y_test.values, predictions):
        all_predictions.append({
            "Model": model_name,
            "Actual Safety Score": round(actual, 4),
            "Predicted Safety Score": round(predicted, 4)
        })

results_df = pd.DataFrame(results)

```

Şekil 4.2.1. Model Karşılaştırması

4.3. Çalıştırma Yönergeleri ve Sistem Konfigürasyonu

Projenin yerel bir bilgisayarda veya sunucuda sorunsuz bir şekilde çalıştırılabilmesi için aşağıdaki kurulum ve konfigürasyon adımlarının sırasıyla izlenmesi gerekmektedir. Proje, bağımlılıkların çakışmaması adına izole bir Python sanal ortamında çalışacak şekilde tasarlanmıştır.

Adım 1: Deponun Klonlanması ve Dizin Geçişi

Öncelikle proje dosyalarının bulunduğu dizine terminal üzerinden giriş yapılmalıdır.

Adım 2: Sanal Ortamın Oluşturulması ve Aktifleştirilmesi

Sistem kütüphaneleriyle çakışmayı önlemek için yeni bir sanal ortam oluşturulmalı ve aktif edilmelidir:

- Windows için: `python -m venv venv` ardından `venv\Scripts\activate`
- macOS/Linux için: `python3 -m venv venv` ardından `source venv/bin/activate`

Adım 3: Gerekli Kütüphanelerin Kurulumu

Projenin çalışması için gereken Pandas, Scikit-learn, XGBoost ve Streamlit gibi kütüphaneler `requirements.txt` dosyasından yüklenmelidir:

- `pip install -r requirements.txt`

Adım 4: Veri Boru Hattının Çalıştırılması

Modelin eğitileceği verilerin hazırlanması için modüller sırasıyla çalıştırılmalıdır:

1. Verinin API'den çekilmesi: `python src/data_collection.py`
2. Verinin temizlenmesi: `python src/data_cleaning.py`
3. Güvenlik skorunun hesaplanması: `python src/safety_score.py`
4. K-Means kümelemesinin yapılması: `python src/clustering.py`
5. Modellerin eğitimi ve karşılaştırması: `python src/model_comparison.py`

Adım 5: Dashboard'un Başlatılması

Tüm veriler işlenip modeller eğitildikten sonra, son kullanıcı arayüzünü ayağa kaldırmak için Streamlit sunucusu başlatılır:

- `streamlit run app.py`

Bu komut çalıştırıldığında terminalde yerel bir URL üretilecek ve sistemin interaktif arayüzü varsayılan web tarayıcısında otomatik olarak açılacaktır.

5. TEST ve DOĞRULAMA

Bu proje, hem veri işleme adımlarını içeren bir yazılım mimarisine hem de tahmine dayalı makine öğrenmesi modellerine sahip olduğu için test süreçleri iki ana başlık altında yürütülmüştür: Yazılım ve veri bütünlüğü doğrulama testleri ile model performans testleri.

5.1. Yazılım ve Veri Bütünlüğü Doğrulaması

Sistemin doğru çalışabilmesi için verinin API'den çekilmesi, temizlenmesi ve güvenlik skoruna dönüştürülmesi aşamalarında çeşitli mantıksal doğrulamalar (validation) yapılmıştır. Bu testlerin temel amacı, hatalı veya eksik verilerin modellere sızmasını engellemek ve sistemin çökmesini önlemektir:

- **Güvenlik Skoru Sınır Testi:** `safety_score.py` modülü çalıştıktan sonra üretilen tüm bölgesel skorların matematiksel olarak kesinlikle 0 ile 100 aralığında kalıp kalmadığı kontrol edilmiştir.
- **Coğrafi Koordinat Doğrulaması:** Veri temizleme aşamasında, eksik (Null) enlem ve boylam değerlerine sahip kayıtlar test edilmiş ve veri setinden izole edilerek Folium haritasının hata vermesi engellenmiştir.
- **Boru Hattı Bütünlük Testi:** Streamlit arayüzünün modeli yeniden eğitmek yerine `outputs/` klasöründeki önceden hesaplanmış `.csv` dosyalarını sorunsuz bir şekilde okuyup okuyamadığı test edilmiş ve sistemin asenkron çalışma bütünlüğü doğrulanmıştır.

5.2. Makine Öğrenmesi Model Testleri ve Sonuçların Değerlendirilmesi

- Projede geliştirilen algoritmaların test edilmesi amacıyla, hazırlanan veri seti, `model_comparison.py` dosyasındaki `train_test_split` fonksiyonu kullanılarak %80 eğitim (train) ve %20 test (test) seti olmak üzere ikiye ayrılmıştır. Eğitim verisiyle öğrenen modeller, daha önce hiç görmedikleri test verisi üzerinde denenmiş ve tahmin başarıları ölçülmüştür.
- Modellerin performansını değerlendirmek için Ortalama Mutlak Hata (MAE), Ortalama Kare Hata (MSE), Kök Ortalama Kare Hata (RMSE) ve Belirleyicilik Katsayısı (R^2 Score) metrikleri kullanılmıştır.

6. SONUÇ

Bu proje, geleneksel suç sayımı yöntemlerinin ötesine geçerek, suç türlerinin ağırlığını hesaba katan ve 0-100 arası normalize edilmiş objektif bir Güvenlik Skoru üretmeyi başarmıştır. Geliştirilen analitik sistem; K-Means algoritması ile bölgeleri risk seviyelerine göre segmentlere ayırmış, Random Forest gibi makine öğrenmesi modelleriyle güvenlik skorunu yüksek doğrulukla tahmin etmiş, LSTM yardımıyla gelecek tahmininde bulunmuş ve elde edilen tüm verileri etkileşimli bir Streamlit arayüzüne entegre ederek karar vericiler için pratik bir araca dönüştürmüştür.

Gelecek fazlarda, modele sosyodemografik verilerin (gelir, eğitim durumu vb.) dahil edilmesi planlanmaktadır. Ayrıca, veri boru hattının gerçek zamanlı çalışacak şekilde otomatikleştirilmesi ve bu web arayüzünün modern araçlarla bir Android uygulamasına dönüştürülerek kullanıcılara konum bazlı anlık risk bildirimleri sunması hedeflenmektedir.

Github: <https://github.com/azraaktas/livability-crime-risk-analysis>

KAYNAKLAR

- [1] City of Chicago. (2024). Crimes - 2001 to Present. Chicago Data Portal. Eriřim Adresi: <https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2>
- [2] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. Eriřim Adresi: <https://scikit-learn.org/>
- [3] Streamlit Inc. (2024). Streamlit Documentation: The fastest way to build and share data apps. Eriřim Adresi: <https://docs.streamlit.io/>
- [4] The pandas development team. (2024). pandas-dev/pandas Eriřim Adresi: <https://pandas.pydata.org/>
- [5] Python Software Foundation. (2024) Eriřim Adresi: <https://www.python.org/>
- [6] Python-Visualization. (2024). Folium: Python Data, Leaflet.js Maps. Resmi Dokümantasyon. Eriřim Adresi: <https://python-visualization.github.io/folium/>
- [7] Towards Data Science. (2020). Interactive Mapping with Folium in Python. Eriřim Adresi: <https://towardsdatascience.com/how-to-generate-interactive-maps-with-folium-b232778758c4/>
- [8] W3Schools. (2023). Machine Learning - K-means Clustering and Normalization. Eriřim Adresi: https://www.w3schools.com/python/python_ml_k-means.asp